

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

Lernziele

- Wie lassen sich Daten in die Datenbank einfügen?
- Wie können diese aktualisiert werden?
- Was gibt es für Möglichkeiten, Daten wieder zu löschen?
- Welche Methoden existieren für die Abfrage von Daten?

Einschub

„Automatisches Hochzählen von IDs“

- In MySQL via auto_increment
- In Oracle mit einem kleinen Umweg:

SEQUENCE auf Primärschlüssel

```
1 CREATE TABLE student (  
2   MatrNr NUMBER(8) NOT NULL,  
3   ...,  
4   PRIMARY KEY (MatrNr) );  
5  
6 CREATE SEQUENCE MatrNrSeq  
   START WITH 10000000;
```

Trigger auf ID

```
1 CREATE OR REPLACE TRIGGER  
   MatrNrTrig  
2 BEFORE INSERT ON student  
3 FOR EACH ROW  
4  
5 BEGIN  
6   SELECT MatrNrSeq.NEXTVAL  
7   INTO   :new.MatrNr  
8   FROM   dual;  
9 END;  
10 /
```

Übersicht

- 1 SQL
 - SELECT Anfragen
 - SQL JOINS
 - SQL Views

Datenmanipulationssprache

① Daten einfügen, löschen und ändern

- **INSERT** – Dateneingabe
- **UPDATE** – Datenaktualisierung
- **DELETE** – Datenlöschung
- u. v. m. **REPLACE, TRUNCATE**

② Daten abfragen

- **SELECT**
- Mengenoperatoren
- **Joins**

Operatoren der relationalen Algebra

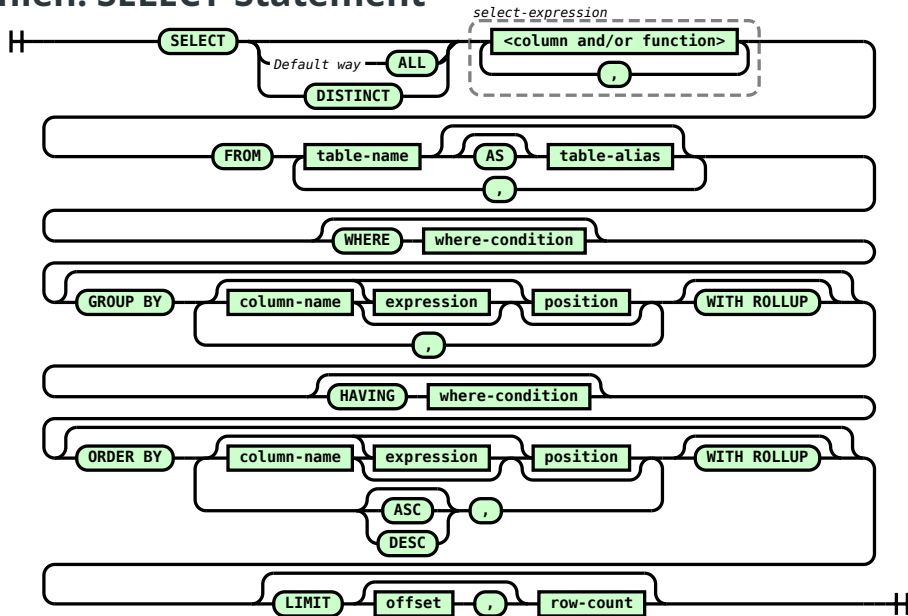
- σ Selektion
- π Projektion
- $\times$ Kreuzprodukt
- $\bowtie$ Join (Verbund)
- ρ Umbenennung
- $-$ Mengendifferenz
- $\div$ Division
- $\cup$ Vereinigung
- $\ltimes$ Semi-Join (links)
- $\rtimes$ Semi-Join (rechts)
- $\Joinleft$ linker äußerer Join
- $\Joinright$ rechter äußerer Join

Einfache SQL-Anfrage

- Projektion (π): **SELECT** .. π .. from
- Selektion (σ): **WHERE** .. σ ..

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Auswählen: SELECT-Statement



Selektieren

Selektion mit WHERE-Bedingung

```
1 SELECT PersNr, Name
2 FROM Professoren
3 WHERE Rang = 'C4';
```

Professoren	
PersNr	Name
2125	Sokrates
2126	Russel
2136	Curie
2137	Kant

SELECT + Sortierung

Sortierung beim SELECT

```
1 SELECT PersNr, Name, Rang
2 FROM   Professoren
3 ORDER BY Rang desc, Name asc;
```

Professoren		
PersNr	Name	Rang
2136	Curie	C4
2137	Kant	C4
2126	Russel	C4
2125	Sokrates	C4
2134	Augustinus	C3
2127	Kopernikus	C3
2133	Popper	C3

SELECT DISTINCT

- Duplikateliminierung:

SELECT DISTINCT

```
1 SELECT DISTINCT Rang
2 FROM Professoren;
```

Professoren
Rang
C4
C3

Anfragen über mehrere Relationen

- Welcher Professor liest „Mäeutik“?
- $\pi_{Name, Titel}(\sigma_{PersNr=gelesenVon \wedge Titel="Mäeutik"}(Professoren \times Vorlesungen))$

SELECT über mehrere Relationen mit WHERE

```
1 SELECT Name, Titel
2 FROM Professoren, Vorlesungen
3 WHERE PersNr = gelesenVon AND Titel = 'Mäeutik';
```

Name	Titel
Sokrates	Mäeutik

Anfragen über mehrere Relationen

- FROM Professoren, Vorlesungen

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
⋮	⋮	⋮	⋮
2137	Kant	C4	7

Vorlesungen			
VorlNr	Titel	SWS	gelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
⋮	⋮	⋮	⋮
5049	Mäeutik	2	2125
⋮	⋮	⋮	⋮
4630	Die 3 Kriterien	4	2137

Anfragen über mehrere Relationen

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5001	Grundzüge	4	2137
2125	Sokrates	C4	226	5041	Ethik	4	2125
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2125	Sokrates	C4	226	5049	Mäeutik	2	2125
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2126	Russel	C4	232	5001	Grundzüge	4	2137
2126	Russel	C4	232	5041	Ethik	4	2125
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2137	Kant	C4	7	4630	Die 3 Kriterien	4	2137

↓ *Auswahl*(σ) ↓ ... WHERE PersNr = gelesenVon
AND Titel = 'Mäeutik'

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5049	Mäeutik	2	2125

↓ *Projektion*(π) ↓ ... SELECT Name, Titel, ...

Name	Titel
Sokrates	Mäeutik

Beispiel Anfragen

- Welche Studierende hören welche Vorlesungen?

SELECT über mehrere Tabellen

```
1 SELECT Name, Titel
2 FROM   Studenten, hoeren, Vorlesungen
3 WHERE  Studenten.MatrNr = hoeren.MatrNr AND
4        hoeren.VorlNr = Vorlesungen.VorlNr;
```

Mengenoperationen

- Mengenoperationen UNION, INTERSECT, MINUS

UNION

```
1 ( SELECT Name
2   FROM Assistenten )
3 UNION
4 ( SELECT Name
5   FROM Professoren );
```

Name
Platon
Aristoteles
Wittgenstein
Rhetikus
Newton
Spinoza
Sokrates
Russel
Kopernikus
Popper
Augustinus
Curie
Kant

SQL-Operatoren

SQL-Operatoren	
Operator	Bedeutung
[NOT] BETWEEN Untergrenze AND Obergrenze	[NOT] TRUE, wenn ein Wert zwischen zwei Werten liegt, einschließlich der Grenzen
[NOT] IN (Wert1, Wert2 ...)	[NOT] TRUE, wenn Übereinstimmung mit irgendeinem Wert der Liste existiert
[NOT] LIKE	[NOT] TRUE, wenn Übereinstimmung mit einem Zeichenmuster existiert; Wildcards: "%" und "_"
IS [NOT] NULL	[NOT] TRUE, wenn ein NULL-Wert vorliegt

Beispiel für Between

Anfrage: Zeige die Namen aller Studierenden, die zwischen dem 01. Januar 1982 und dem 31. Dezember 1983 geboren wurden.

SELECT Between

```
1 SELECT Name
2 FROM   Studenten
3 WHERE  Geburtstag BETWEEN '1982-01-01' AND '1983-12-31';
```

ist äquivalent zu

```
1 SELECT Name
2 FROM   Studenten
3 WHERE  Geburtstag >= '1982-01-01'
4 AND    Geburtstag <= '1983-12-31';
```

Mengenoperationen

- Mengenoperationen: Existenzquantor **exists**
- Anfrage: Liefere die Namen aller Professoren, die KEINE Vorlesung halten

Select mit exists sowie einer Unterabfrage

```
1 SELECT p.Name
2 FROM Professoren p
3 WHERE NOT exists ( SELECT *
4                     FROM Vorlesungen v
5                     WHERE v.gelesenVon = p.PersNr );
```

Mengenvergleich

- Unkorrelierte Unteranfrage: meist effizienter, wird nur einmal ausgewertet

Unteranfrage

```
1 SELECT Name
2 FROM Professoren
3 WHERE PersNr NOT IN ( SELECT gelesenVon
4                       FROM Vorlesungen );
```

Operatoren

Operatoren werden benutzt, um Daten zu verknüpfen und ein Resultat auszugeben.

Operator	Beschreibung	Beispiel
+ -	Positiver oder negativer Ausdruck	SELECT * FROM Teile WHERE - Bestand < 0
* /	Multiplikation Division	UPDATE Angestellte SET Gehalt = Gehalt * 1.1;
+ -	Addition Subtraktion	SELECT Gehalt - Abzuege FROM Angestellte;
	Konkatenation ¹	SELECT Vorname ' ' Nachname FROM Angestellte;

Bei Oracle wird die Spalte leer angezeigt und bei MySQL mit der Kennung NULL.

¹Pipes müssen in MySQL erst freigeschaltet werden; ansonsten mit CONCAT(str1, str2, ...) arbeiten

Datenmanipulationssprache

- Der Vergleich mit **ALL**
- Anfrage: Liefere die Namen aller Studierenden, die am längsten (bzgl. Semesteranzahl) studieren

SELECT ALL

```
1 SELECT Name
2 FROM Studenten
3 WHERE Semester >= all ( SELECT Semester
4                           FROM Studenten );
```

Aggregatfunktion und Gruppierung

Mit **GROUP BY** werden die Ergebnisdatensätze nach der **WHERE**-Klausel in Gruppen eingeteilt, so dass innerhalb der Gruppen die Werte der **GROUP BY**-Spalten gleich sind.

Bsp.: Tabelle Professoren und Gruppe aller Professoren mit gleichem Rang.

SQL-Gruppenfunktionen	
Funktion	Bedeutung
COUNT	Anzahl der Werte in einer Spalte
SUM	Summe der Werte in einer Spalte
AVG	Mittelwert der Spalte
MIN / MAX	Kleinster / Größter Wert der Spalte

Aggregatfunktion und Gruppierung

- SQL erzeugt pro Gruppe ein Ergebnistupel
- Alle in der **SELECT**-Klausel aufgeführten Attribute - außer den aggregierten - müssen auch in der **GROUP BY**-Klausel aufgeführt werden
- Dadurch kann SQL sicherstellen, dass sich das Attribut nicht innerhalb der Gruppe ändert

SELECT Durchschnitt an Semestern für alle Studierende

```
1 SELECT avg (Semester)
2 FROM   Studenten;
3
4 /* Ergebnis: 7,625 */
```


Aggregatfunktion und Gruppierung

- Syntax der Gruppenbildung:
- `SELECT ... FROM ... WHERE ... GROUP BY col1 [col2,...]`

Beispiel für Summe

```
1 SELECT    gelesenVon , sum (SWS)
2 FROM      Vorlesungen
3 GROUP BY  gelesenVon;
```

- Alternativ (Umbenennung ρ):

Umbenennung der Summenfunktion

```
1 SELECT    gelesenVon , sum (SWS) as SummeSWS
2 FROM      Vorlesungen
3 GROUP BY  gelesenVon;
```

Aggregatfunktion und Gruppierung

- Beispiel für Gruppen innerhalb von Gruppen:
- Wir zählen, wie oft innerhalb eines Namens gleiche Vornamen vorkommen

Group By

```
1 SELECT name, vname, COUNT(name)
2 FROM t_ma
3 GROUP BY name, vname;
```

Aggregatfunktion und Gruppierung

- Die Gruppen können mittels einer Bedingung selektiert werden (HAVING-Klausel)
- Bsp. 2:

Having

```
1 SELECT gelesenVon, Name, sum(SWS)
2 FROM Vorlesungen, Professoren
3 WHERE gelesenVon = PersNr AND Rang = 'C4'
4 GROUP BY gelesenVon, Name
5 HAVING avg(SWS) >= 3;
```

Aggregatfunktion und Gruppierung

Syntax

```
1 SELECT ... FROM tabelle(n)
2 [GROUP BY Datenfeld,...] [HAVING Bedingung]
```

- Syntaxzusammenfassung:
- Anmerkungen:
 - Nur innerhalb der **HAVING**-Klausel sind Aggregatfunktionen wie **COUNT()** erlaubt, nicht in der **WHERE**-Klausel.
 - Gruppen können kaskadiert werden, d. h. innerhalb einer Gruppe können weitere Gruppen gebildet werden (u. s. w.).
 - Wie? Durch Angabe mehrerer Datenfelder hinter **GROUP BY**

Single-Row-Funktionen auf Zeichenketten

Funktion	Beschreibung	Beispiel
LOWER (String)	Konvertiert String in Kleinbuchstaben	SELECT LOWER (Nachname) FROM Angestellte;
UPPER (String)	Konvertiert String in Großbuchstaben	SELECT UPPER(Nachname) FROM Angestellte;
LENGTH (String)	Liefert die Anzahl der Zeichen des Strings	SELECT LENGTH (Nachname) FROM Angestellte;
SUBSTR (String, m [n])	Liefert eine Zeichenkette der Länge n ab der Position m	SELECT SUBSTR(Nachname, 2,2) FROM Angestellte;
CONCAT (String1, String2 ...)	Hängt zwei Strings aneinander	SELECT CONCAT('Hugo', 'Schmidt') FROM DUAL;
ASCII (String)	Wandelt ein einzelnes Zeichen in ASCII-Code um	SELECT ASCII('F') FROM DUAL;
CHR (ascii)	Wandelt ASCII-Code in Zeichen	SELECT CHR(70) FROM DUAL;
TRIM (String)	Entfernt Suffixe und Präfixe	SELECT TRIM('H' FROM 'Hugo') FROM DUAL
(L R)PAD (String, Länge, Füllaus- druck)	Füllt Zeichenketten auf eine bestimm- te Länge von links bzw. rechts mit Füll- zeichen auf	SELECT LPAD('Hugo Müller', 25, 'x') FROM DUAL;

Numerische und mathematische Single-Row-Funktionen

Eine Auswahl von Operatoren für den numerischen und mathematische Funktionen

Funktion	Beschreibung	Beispiel
ROUND (Ausdruck, n)	Rundet den Ausdruck auf n Dezimalstellen	SELECT ROUND(Gehalt, 2) FROM Angestellte;
MOD (m,n)	Liefert den Rest von m dividiert durch n	SELECT MOD (Gehalt, 2) FROM Angestellte;
TRUNCATE (Nummer, n)	Schneidet die Nummer nach n Dezimalstellen ab (Negative Werte schneiden vor dem Komma ab)	SELECT TRUNCATE(3.1415, 2) => 3.14
NVL ² IFNULL ³ (Spalte, Wert)	Ersetzt einen NULL-Wert in der Spalte durch den angegebenen Wert, sonst wird der Wert der Spalte zurückgegeben	SELECT {NVL IFNULL} (Gehalt, 0) FROM Angestellte;
ABS, COS, SIN, LN, LOG, SIGN, POWER, SINH, SQRT, TAN	Verschiedene mathematische Funktionen, siehe Hersteller-Originalliteratur	

²Oracle-spezifisch

³Entsprechende MySQL Funktion

Single-Row Funktionen für Datum

Funktion	Beschreibung	Beispiel
ADDDATE (datum, Tage) ADDDATE (datum, INTERVAL wert Einheit)	Fügt einem Datum ein Zeit- / Datumintervall hinzu	SELECT ADDDATE("2017-06-15", INTERVAL 10 DAY);
CURRENT_DATE (), CURRENT_TIME (), CURRENT_TIMESTAMP	Gibt das aktuelle Datum / die aktuelle Uhrzeit / den aktuellen Zeitstempel zurück	SELECT current_timestamp();
DATEDIFF (datum1, datum2)	Differenz zwischen datum1 - datum2 in Tagen.	SELECT DATEDIFF("2017-06-25", "2017-06-15");
DATE_FORMAT (datum, format)	Formatiert das gegebene Datum mit diversen Formatierungsplatzhaltern ⁴ .	SELECT DATE_FORMAT("2017-06-15", "%W %M %e %Y");
DAY (datum) DAYOFMONTH (datum)	Beide Funktionen liefern den Tag im Monat.	SELECT dayofmonth(current_date());

⁴https://www.w3schools.com/sql/func_mysql_date_format.asp

Der LIKE - Operator

Der **LIKE**-Operator dient zum Abfragen von Textmustern über Schablonen und Wildcards. Als Wildcards sind die definierten Symbole **%** und **_** zulässig.

% steht für kein, ein oder mehrere Zeichen

_ für ein beliebiges Zeichen

LIKE %

```
1 SELECT *  
2 FROM Studenten  
3 WHERE Name LIKE 'M%';
```

LIKE _

```
1 SELECT *  
2 FROM Studenten  
3 WHERE Name LIKE 'M_ier';
```


Der IN - Operator

Mit dieser Bedingung wird getestet, ob ein Vergleichswert in einer Menge von Werten enthalten ist. Ist der Ausdruck NULL, so ist das Ergebnis UNKNOWN.

IN

```
1 SELECT *  
2 FROM Studenten  
3 WHERE Semester IN (8,10);
```

Limitierungen

- Limitiert die Ergebniszeilen auf entweder:
 - von Anfang bis n
 - oder von einem offset bis n
- Syntax: LIMIT {[offset,] row_count}

SELECT LIMIT

```
1 SELECT * FROM tbl LIMIT 5,10;  -- Gibt die Reihen 6-15 aus
```

Mehrere Relationen

- Falls mehrere Relationen in der **FROM**-Klausel auftauchen, werden sie mit einem Kreuzprodukt verbunden
- Anfrage: Gib alle Vorlesungen und Professoren aus.

SELECT mehrere Relationen

```
1 SELECT *  
2 FROM vorlesungen , professoren ;
```

VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
4052	Logik	4	2125	2125	Sokrates	C4	226
4052	Logik	4	2125	2126	Russel	C4	232
4052	Logik	4	2125	2127	Kopernikus	C3	310
4052	Logik	4	2125	2133	Popper	C3	52
4052	Logik	4	2125	2134	Augustinus	C3	309
4052	Logik	4	2125	2136	Curie	C4	36
4052	Logik	4	2125	2137	Kant	C4	7
4630	Die 3 Kritiken	4	2137	2125	Sokrates	C4	226
4630	Die 3 Kritiken	4	2137	2126	Russel	C4	232
4630	Die 3 Kritiken	4	2137	2127	Kopernikus	C3	310
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Übersicht

1 SQL

SELECT Anfragen

SQL JOINS

SQL Views

JOINS (1)

- Kreuzprodukte machen meistens keinen Sinn, interessanter sind JOINS!
- JOIN-Prädikate werden in der **WHERE**-Klausel angegeben:

× -> JOIN

```
1 SELECT *  
2 FROM vorlesungen , professoren  
3 WHERE gelesenVon = PersNr;
```

VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
4052	Logik	4	2125	2125	Sokrates	C4	226
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7
5001	Grundzuege	4	2137	2137	Kant	C4	7
5022	Glaube und Wissen	2	2134	2134	Augustinus	C3	309
5041	Ethik	4	2125	2125	Sokrates	C4	226
5043	Erkenntnistheorie	3	2126	2126	Russel	C4	232
5049	Mäeutik	2	2125	2125	Sokrates	C4	226
5052	Wissenschaftstheorie	3	2126	2126	Russel	C4	232
5216	Bioethik	2	2126	2126	Russel	C4	232
5259	Der Wiener Kreis	2	2133	2133	Popper	C3	52

JOINS (2)

- Es dürfen beliebig viele Entitäten in der **FROM**-Klausel stehen
- Wenn keine Kreuzprodukte erwünscht, sollten alle in der **WHERE**-Klausel gejoint werden
- Die verschiedenen JOIN-Varianten aus der relationalen Algebra sind auch in SQL möglich:

JOIN-Varianten

```
1 SELECT *  
2 FROM R1 [cross | inner | natural | left outer | right outer | full outer]  
3 JOIN R2 [ON R1.A = R2.B];
```

Joins (3)

- Weiteres Problem: Namenskollisionen (gleichnamige Attribute in verschiedenen Relationen) müssen aufgelöst werden.
- Beispiel: JOIN von
 - Studenten (MatrNr, Name, Geburtstag)
 - hoeren (MatrNr, VorlNr)
 - Vorlesungen (VorlNr, Titel)

Qualifizierte Attributnamen

- In dieser Beispielanfrage muss spezifiziert werden, woher *MatrNr* und *MatrNr* herkommen sollen
- Dazu schreibt man den Relationenname vor den Attributnamen

Attribute benennen

```
1 SELECT *
2 FROM   studenten , hoeren , vorlesungen
3 WHERE  studenten.MatrNr = hoeren.MatrNr
4 AND    hoeren.VorlNr    = vorlesungen.VorlNr;
```


- Um sich Tipparbeit zu sparen, können die Relationen auch umbenannt werden

Zeit ist Geld...

```
1 SELECT *
2 FROM studenten S, hoeren h, vorlesungen V
3 WHERE S.MatrNr = h.MatrNr
4 AND h.VorlNr = V.VorlNr;
```

Joins in SQL

- to join: etwas verbinden, d. h. mit JOINS werden Tabellen miteinander verbunden
- Arten:
 - **CROSS JOIN**: Kartesisches Produkt
 - **INNER JOIN**: Vergleich zweier oder mehrerer Spalten (Equi-Join, Theta-Join, Natural-Join)
 - **OUTER JOIN**: Erhält alle Datensätze einer oder mehrerer Tabellen, unabhängig von Referenz in referenzierter Tabelle (Left-Outer-Join, Right-Outer-Join, Full-Outer-Join)
 - **SELF JOIN**: Verknüpfung mit der selben Tabelle

Inner-Join

- Anstelle von CROSS JOIN ...
- ... besser **INNER JOIN** mit Bedingung

INNER JOIN

```
1 /* Mit Eingabe einer Bedingung */
2 SELECT * FROM professoren INNER JOIN pruefen ON professoren.PersNr
   = pruefen.PersNr;
3
4 /* Oder bei gleichnamigen Attributen über NATURAL JOIN */
5 SELECT * FROM professoren NATURAL JOIN pruefen;
```

Outer-Join

- Unterschied zu Inner-Join: Null wird berücksichtigt
- Es werden alle Datensätze einer der beteiligten Tabellen in die Ergebnisrelation übernommen, ohne dass verbundene Datensätze in der anderen Tabelle existieren.
- Attribute der anderen Tabelle enthalten ggf. NULL-Werte.
- 3 Unterarten von Outer-Joins:
 - Left Outer Join
 - Right Outer Join
 - Full Join

Left Outer Join

- Alle Datensätze der linken Tabelle bleiben erhalten
- Anfrage: Professoren mit und ohne Prüfungen

SELECT: Left Outer Join

```
1 SELECT *
2 FROM professoren
3 LEFT OUTER JOIN pruefen
4 ON professoren.PersNr = pruefen.PersNr
```

PersNr	Name	Rang	Raum	MatrNr	VorlNr	PersNr	Note
2125	Sokrates	C4	226	25403	5041	2125	2.0
2126	Russel	C4	232	28106	5001	2126	1.0
2127	Kopernikus	C3	310	NULL	NULL	NULL	NULL
2133	Popper	C3	52	NULL	NULL	NULL	NULL
2134	Augustinus	C3	309	NULL	NULL	NULL	NULL
2136	Curie	C4	36	NULL	NULL	NULL	NULL
2137	Kant	C4	7	27550	4630	2137	2.0

Right Outer Join

- Alle Datensätze der rechten Tabelle bleiben erhalten
- Anfrage: Professoren mit und ohne Assistenten

SELECT: Right Outer Join

```
1 SELECT *  
2 FROM assistenten  
3 RIGHT OUTER JOIN professoren ON professoren.PersNr = assistenten.  
   Boss  
4 LIMIT 0 , 30;
```

Full Outer Join

- Alle Datensätze beider Tabelle bleiben erhalten
- Wird in einigen DBMS nicht unterstützt (z. B. in MySQL)
- Alternative:

SELECT: Full Outer Join

```
1 SELECT a.name as Assi, p.name as Prof
2 FROM assistenten a
3 RIGHT OUTER JOIN professoren p ON p.PersNr = a.Boss
4
5 UNION
6
7 SELECT a.name as Assi, p.name as Prof
8 FROM assistenten a
9 LEFT OUTER JOIN professoren p ON p.PersNr = a.Boss
```

Übersicht

1 SQL

SELECT Anfragen

SQL JOINS

SQL Views

Sichten (Views)

- Sicht = Vordefinierte Abfrage (Ext. Sicht)
- Prinzip: Eine **SELECT**-Abfrage wird unter einem Namen X „fest“ in der DB etabliert und steht danach unter diesem Namen zur Verfügung für
 - Wiederholung der Abfrage
 - Konstruktion neuer Abfragen auf Basis von X

View

```
1 CREATE VIEW StudentenHoherSemester AS SELECT *
2 FROM studenten
3 WHERE semester >= 10
```

Sichten (Views)

- Sichten sind nützlich
 - um einfacher auf komplexe Abfragen zugreifen zu können (Shortcut)
 - Um den Zugriff auf Daten einzuschränken (Datenschutz)
 - Bsp.: Benutzer X darf nur die Spalten Namen und Vornamen, nicht aber Gehalt von Tabelle Personal sehen; er hat daher nur das Recht, auf die entsprechende Sicht lesend zuzugreifen
- Daten in Sichten können unter bestimmten Umständen auch geändert werden (wenn eine eindeutige Zuordnung zu den Ursprungstabellen möglich)

Zusammenfassung (1)

Operationen	Umsetzung in der SQL-SELECT-Anweisung
Projektion (π)	Spaltenliste der SELECT-Klausel SELECT Spalte_3, Spalte_2 FROM Tabelle
Selektion (σ)	WHERE-Bedingung SELECT * FROM Tabelle WHERE A > B;
Kartesisches Produkt ($\times$)	FROM-Klausel ohne WHERE-Klausel SELECT Tabelle_1.*, Tabelle_2.* FROM Tabelle_1, Tabelle_2;
Vereinigung ($\cup$)	UNION (für zwei Tabellen mit gleicher Spaltendefinition) SELECT * FROM Tabelle_1 UNION SELECT * FROM Tabelle_2;
Differenz ($-$)	EXCEPT oder MINUS SELECT * FROM Tabelle_1 EXCEPT SELECT * FROM Tabelle_2;
Durchschnitt ($\cap$)	INTERSECT SELECT * FROM Tabelle_1 INTERSECT SELECT * FROM Tabelle_2;

Zusammenfassung (2)

Operationen	Umsetzung in der SQL-SELECT-Anweisung
NATURAL JOIN ($\bowtie$)	NATURAL JOIN-Operator: Gleichnamige Attribut-Paare mit demselben Datentyp aus verbundenen Tabellen werden auf Gleichheit verglichen. Ergebnis wird auf eine Spalte reduziert. ON-Befehl bei NATURAL JOIN nicht verwenden. Beispiel: SELECT Artikel.TNr, AuftragsNr, Menge FROM Artikel NATURAL JOIN Auftragspositionen
THETA-JOIN ($\bowtie_{\theta}$)	JOIN-Operator mit beliebiger Bedingung oder: Für die Tabellen der FROM-Klausel kann eine beliebige WHERE-Bedingung formuliert werden. SELECT k.Kun_Nr, Nachname, Bestelldatum FROM Auftraege a, Kunden k WHERE k.Kun_Nr = a.Kun_Nr AND Bestelldatum IS NOT NULL;
OUTER-JOIN ($\Join$ $\Join_{\text{full}}$)	LEFT, RIGHT, FULL OUTER-JOIN-Operator SELECT k.Kun_Nr, Nachname, Bestelldatum FROM Auftraege a RIGHT OUTER JOIN Kunden k ON k.Kun_Nr = a.Kun_Nr;

Sie wissen nun

- ... was eine Relationale Anfragesprache ist
- ... was Beispiele für Relationale Anfragesprachen sind
- ... was eine DDL und eine DML ist
- ... SQL ist und aus welchen Teilsprachen SQL besteht
- ... wie man in SQL Datenbanken und Tabellen anlegt und wieder löscht
- ... wie man in SQL Daten speichert, ändert und löscht
- ... wie man mit SQL die Operationen der relationalen Algebra umsetzt
- ... wie man mit SQL Aggregation, Gruppierung und geschachtelte Anfragen sowie Sichten programmiert

Vielen Dank!